

GitHub 工程案例 · 2026 年 9 月

用 Copilot, 迁移到 Rust

从一场大型重写，看 AI 编程的工程边界

80 万+ 生产代码行 128 迁移 PR

原文: Stephen Toub · GitHub Blog · 2026-09-16

先问部署需求，再选语言

应用与 SDK



CLI 子进程



Node.js / V8

共享运行时，也会共享它的成本。

启动

加载与初始化

内存

每个实例的开销

集成

跨进程通信

案例事实：原架构通过 CLI 子进程承载运行时。

同一引擎，两种承载方式

进程内

应用 → FFI → Rust

减少进程边界

与应用共享故障边界

进程外

应用 → 通信 → Rust 服务

保留独立进程

仍需通信与进程管理

Rust 重写完成 ≠ 所有客户端默认进程内运行

让每一步都能交付

按组件替换，持续发布。



先处理依赖较少的部分，再靠近有状态的核心。

案例事实：原地逐组件替换；小步迁移期间保持主分支可发布。

把任务写成一份验收契约

“把这个模块
改成 Rust”

仍然留下太多空白

范围 入口、依赖、回调、状态

行为 正常、失败、取消、恢复

证据 测试结果与差异说明

完成 旧路径与临时桥接清理

能编译，只回答了一部分问题

01

结构成立

类型与接口匹配

02

行为一致

输入与输出契约

03

系统可用

时序、取消与资源

独立的测试标准，是迁移的参照物。

案例启示：编译器接受的程序，仍可能出现行为回归。

内存收益，放回测试场景中看

十客户端批次 · 相对基线增加的驻留私有内存峰值

迁移前

1,383

MB · Node.js

Rust 进程外

247

MB

Rust 进程内

126

MB

本地确定性响应；排除模型推理与网络延迟。
交付系统的前后对比，不能归因于语言单一变量。

数据来源：原文性能章节。真实应用的收益取决于工作负载。

代码生成之后，工作仍在继续

约 12 万美元

原文披露的 token 支出
不等于项目完整成本

架构与任务拆分

构建、测试与审查

团队集成与发布判断

衡量“可验证的交付”，比统计生成行数更有用。

第一次试点，可以这样设计

选小组件 边界清晰，外部依赖少

冻结参照 先记录接口、样例与失败行为

跑完整链路 构建 → 接入 → 测试 → 发布

再做决定 按缺陷与资源收益决定是否扩大

用试点发现真实成本，也检验团队是否能接手维护。

本片提出的试点方法；不是对所有项目的语言迁移建议。

结语

AI 扩大执行能力， 工程师定义完成标准。

目标清楚 · 边界清楚 · 证据清楚

案例摘述与独立工程解读

Stephen Toub · GitHub Blog · 2026-09-16

原文: [Migrating the GitHub Copilot runtime to Rust, using Copilot](#)